Práctica SQL — Base de Datos Baloncesto

Explicación detallada de los 16 ejercicios DDL y DML

Este documento explica, comando a comando, cada una de las 16 operaciones realizadas sobre la base de datos **Baloncesto**. Se incluyen los fundamentos teóricos, la sintaxis utilizada y el comportamiento esperado por parte del motor MySQL.

Ejercicio 1 — Crear la tabla Entrenadores

Se crea la tabla **Entrenadores** con los campos requeridos: **id_entrenador** (PK autoincremental), **nombre** y **edad** (NOT NULL), **especialidad** (opcional) e **id_equipo** como clave foránea que referencia a *Equipos(id_equipo)*.

```
CREATE TABLE Entrenadores ( id_entrenador INT PRIMARY KEY AUTO_INCREMENT, nombre
VARCHAR(50) NOT NULL, edad INT NOT NULL, especialidad VARCHAR(50), id_equipo INT,
CONSTRAINT fk_entrenadores_equipo FOREIGN KEY (id_equipo) REFERENCES Equipos(id_equipo)
ON DELETE SET NULL ON UPDATE CASCADE );
```

Justificación de ON DELETE SET NULL

Se eligió **ON DELETE SET NULL** porque si se elimina un equipo, el entrenador sigue existiendo como persona en la organización y sus datos (historial, salario…) deben conservarse. Simplemente se deja sin equipo asignado (*id_equipo* = *NULL*). Las alternativas descartadas son:

• **ON DELETE CASCADE**: eliminaría automáticamente al entrenador si se borra su equipo. Inaceptable porque se perdería información valiosa.
• **ON DELETE RESTRICT**: impediría borrar un equipo mientras tenga entrenadores. Demasiado restrictivo para la gestión habitual.
• **ON DELETE NO ACTION**: comportamiento equivalente a RESTRICT en MySQL.

ON UPDATE CASCADE garantiza que si cambia el id de un equipo (raro pero posible), la FK en Entrenadores se actualiza automáticamente.

Ejercicio 2 — Añadir columna salario con restricción

ALTER TABLE … ADD COLUMN permite añadir nuevas columnas a una tabla existente sin necesidad de recrearla. Se añade la columna **salario** con una restricción CHECK que impide valores negativos.

```
ALTER TABLE Entrenadores ADD COLUMN salario DECIMAL(10,2), ADD CONSTRAINT
chk_entrenadores_salario CHECK (salario >= 0);
```

El tipo **DECIMAL(10,2)** almacena hasta 10 dígitos significativos con 2 decimales, adecuado para valores monetarios. La constraint *CHECK* rechaza cualquier inserción o actualización que intente asignar un salario negativo.

Ejercicio 3 — Modificar el tipo de la columna salario

ALTER TABLE … MODIFY COLUMN cambia la definición de una columna existente (tipo, nulidad, valor por defecto…) sin alterar el resto de la estructura.

```
ALTER TABLE Entrenadores MODIFY COLUMN salario DECIMAL(10,2);
```

En este caso el tipo ya era DECIMAL(10,2) desde el ejercicio anterior, por lo que la operación confirma y mantiene esa definición. En un escenario real podría usarse para cambiar, por ejemplo, de INT a DECIMAL cuando el negocio lo requiera.

Ejercicio 4 — Crear la tabla Prueba

CREATE TABLE define una nueva tabla en la base de datos. Se crea **Prueba** con dos columnas de ejemplo: un identificador autoincremental y una descripción de texto obligatoria.

```
CREATE TABLE Prueba ( id_prueba INT PRIMARY KEY AUTO_INCREMENT, descripcion VARCHAR(100)
NOT NULL, fecha_alta DATE );
```

Ejercicio 5 — Eliminar completamente la tabla Prueba

DROP TABLE elimina la tabla y todos sus datos de forma permanente. A diferencia de TRUNCATE, también borra la propia definición de la tabla (estructura, índices, restricciones). La operación es irreversible.

```
DROP TABLE Prueba;
```

Nota: DROP TABLE no puede deshacerse con ROLLBACK si la sesión no está dentro de una transacción explícita, y en MySQL DDL es auto-commit por defecto.

Ejercicio 6 — Vaciar Entrenadores sin eliminar su estructura

TRUNCATE TABLE elimina todos los registros de la tabla pero conserva su estructura (columnas, índices, restricciones). Es más eficiente que DELETE sin WHERE porque no genera log fila a fila y reinicia el contador AUTO_INCREMENT.

```
TRUNCATE TABLE Entrenadores;
```

Diferencias clave con DELETE:
• TRUNCATE no activa triggers de DELETE.
• TRUNCATE no puede filtrarse con WHERE (siempre vacía toda la tabla).
• TRUNCATE reinicia el AUTO_INCREMENT a 1.

Ejercicio 7 — Insertar un entrenador

INSERT INTO … VALUES añade una nueva fila a la tabla. Se especifican explícitamente las columnas para mayor claridad y seguridad.

```
INSERT INTO Entrenadores (nombre, edad, especialidad, id_equipo, salario) VALUES ('Luis
Fernandez', 45, 'Ataque', 1, 3500.00);
```

El campo *id_entrenador* se omite porque es AUTO_INCREMENT: MySQL le asigna automáticamente el siguiente valor disponible.

Ejercicio 8 — Insertar tres entrenadores en una sola sentencia

MySQL permite insertar múltiples filas en una única sentencia INSERT, lo que mejora el rendimiento al reducir el número de viajes al servidor.

```
INSERT INTO Entrenadores (nombre, edad, especialidad, id_equipo, salario) VALUES ('Maria
Gomez', 38, 'Defensa', 2, 2800.00), ('Pedro Sanchez', 52, 'Preparacion fisica', 3,
4100.00), ('Ana Martinez', 41, 'Tactica general', 4, 3200.00);
```

Ejercicio 9 — Insertar entrenador con equipo inexistente

Se intenta insertar un entrenador con *id_equipo* = *999*, valor que no existe en la tabla **Equipos**.

```
-- Esta sentencia produce ERROR (se muestra comentada): -- INSERT INTO Entrenadores
(nombre, edad, especialidad, id_equipo, salario) -- VALUES ('Entrenador Fantasma', 35,
'Ninguna', 999, 1000.00);
```

Nota: MySQL rechaza la operación con ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails. La FK fk_entrenadores_equipo exige que id_equipo exista en Equipos. Dado que 999 no existe, la inserción se cancela y la tabla no se modifica.

Ejercicio 10 — Actualizar el salario de un entrenador concreto

UPDATE … SET … WHERE modifica uno o más campos de las filas que cumplen la condición especificada. La cláusula **WHERE** es fundamental para no afectar registros no deseados.

```
UPDATE Entrenadores SET salario = 5000.00 WHERE id_entrenador = 1;
```

Solo se actualiza la fila cuyo id_entrenador sea 1. Si el CHECK constraint detectara un valor negativo, MySQL rechazaría la operación.

Ejercicio 11 — Incrementar el salario de todos los entrenadores en 200

Al omitir la cláusula WHERE, el UPDATE afecta a **todas las filas** de la tabla. Se puede referenciar el valor actual de la columna en la expresión del SET.

```
UPDATE Entrenadores SET salario = salario + 200;
```

MySQL evalúa *salario + 200* para cada fila de forma individual, por lo que cada entrenador recibe exactamente 200 unidades adicionales independientemente de su salario previo.

Ejercicio 12 — Eliminar un entrenador concreto

DELETE FROM … WHERE elimina las filas que cumplen la condición. La clave primaria garantiza que solo se elimine un registro.

```
DELETE FROM Entrenadores WHERE id_entrenador = 2;
```

Si Entrenadores tuviera tablas hijas con FK apuntando a ella y con ON DELETE RESTRICT, la operación podría fallar. En este esquema la tabla no tiene hijos, por lo que la eliminación se ejecuta sin problema.

Ejercicio 13 — UPDATE sin WHERE — SET salario = 3000

Se ejecuta un UPDATE masivo sin cláusula WHERE:

```
UPDATE Entrenadores SET salario = 3000;
```

¿Qué ocurre? MySQL actualiza el campo *salario* de **todos** los registros de la tabla al valor 3000, sin excepción. No importa cuál era el salario previo de cada entrenador: todos quedan igualados a 3000.

Si el modo **safe updates** (--safe-updates o sql_safe_updates) está activado, MySQL podría bloquear esta sentencia al detectar que no incluye una condición sobre la clave primaria, lanzando un error de protección.

Ejercicio 14 — DELETE FROM sin WHERE — operación peligrosa

Se ejecuta un DELETE sin ninguna condición:

```
DELETE FROM Entrenadores;
```

¿Qué ocurre? Se eliminan **todos** los registros de la tabla Entrenadores. La estructura (columnas, índices, restricciones) permanece intacta, pero queda vacía.

¿Por qué es peligrosa?
1. **Sin filtro:** afecta a cada fila de la tabla, sin distinción.
2. **Irreversible en producción:** si no hay una transacción activa con ROLLBACK preparado, los datos se pierden de forma definitiva.
3. **Error humano:** olvidar el WHERE es un error frecuente con consecuencias catastróficas en entornos reales.
4. **Diferencia con TRUNCATE:** DELETE sin WHERE genera log de transacción fila a fila (más lento y voluminoso) y dispara triggers, pero el resultado es equivalente: tabla vacía.

Nota: Buena práctica: siempre testear con un SELECT antes de ejecutar un DELETE o UPDATE masivo, y usar transacciones explícitas (BEGIN/ROLLBACK/COMMIT).

Ejercicio 15 — Eliminar todos los registros de Entrenadores

Para demostrar la operación de vaciado completo se insertan primero dos entrenadores y se eliminan con TRUNCATE, que es la forma más eficiente de vaciar una tabla.

```
INSERT INTO Entrenadores (nombre, edad, especialidad, id_equipo, salario) VALUES
('Carlos Ruiz', 50, 'Ataque', 1, 3000.00), ('Lucia Torres', 44, 'Defensa', 5, 2700.00);
TRUNCATE TABLE Entrenadores;
```

TRUNCATE es equivalente a DROP + CREATE en cuanto al resultado, pero preserva los permisos y definición de la tabla. Además reinicia el AUTO_INCREMENT, lo que puede ser relevante para el control de identificadores.

Ejercicio 16 — Eliminar un equipo con jugadores asociados

Se intenta eliminar el equipo con id 1 (Lions), que tiene jugadores y partidos asociados:

```
-- Produce ERROR (se muestra comentada): -- DELETE FROM Equipos WHERE id_equipo = 1;
```

Nota: MySQL lanza ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constraint fails. La operación es bloqueada.

¿Por qué ocurre? La tabla **Jugadores** declara la FK *fk_jugadores_equipo* sobre *id_equipo* con **ON DELETE SET NULL**. En este caso concreto no bloqueará, pero la tabla **Partidos** declara dos FKs (*fk_partidos_equipo_local* y *fk_partidos_equipo_visitante*) con **ON DELETE RESTRICT**, que sí impide el borrado.

¿Qué relación lo impide exactamente?
Las claves foráneas de *Partidos.equipo_local* y *Partidos.equipo_visitante* que referencian a *Equipos(id_equipo)* con ON DELETE RESTRICT.

¿Cómo solucionarlo? Antes de borrar el equipo habría que:
1. Eliminar o reasignar todos los partidos donde el equipo sea local o visitante.
2. Reasignar o eliminar los jugadores del equipo (aunque ON DELETE SET NULL lo haría automáticamente).
3. Una vez sin registros dependientes, el DELETE en Equipos se ejecutaría correctamente.